Optimal Stopping with Signatures

C. Bayer, P. Hager, S. Riedel, J. Schoenmakers

arXiv:2105.00778 — Math. PR, May 2021

Alexis Schneider

ENSIIE

April 29 2026

Outline

The Optimal Stopping Problem

Central Problem

Compute $V^* = \sup_{\tau \in \mathcal{S}} \mathbb{E}[Y_{\tau \wedge T}]$,
where Y is adapted to $(\mathcal{F}_t)$ generated by a rough path X, and $\mathcal{S}$ is the set of all $(\mathcal{F}_t)$-stopping times.

Financial reading

- $Y_t = e^{-rt}(K - S_t)^+$: discounted payoff of an American put
- τ: optimal exercise date
- V^*: fair price of the option

Key challenge

- Classic methods (DP) require X to be *Markov*
- fBm with $H \neq \frac{1}{2}$: **not** a semimartingale, **not** Markovian
- Path-dependence is *essential* here

Main contribution

A **model-free** framework based on *path signatures* that handles any continuous (geometric) random rough path, including fractional Brownian motion.

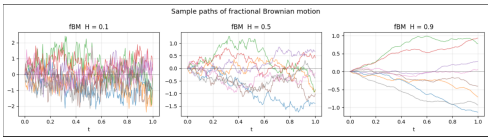
Why Is This Hard? The Non-Markovian Case

Fractional Brownian motion (fBm)

$$\mathbb{E}[X_s X_t] \;=\; \tfrac{1}{2}\big(|s|^{2H} + |t|^{2H} - |t-s|^{2H}\big)$$

- $H = \frac{1}{2}$: standard BM (Markov)
- $H < \frac{1}{2}$: *rough*, anti-persistent paths
- $H > \frac{1}{2}$: *smooth*, long-memory paths
- $H \neq \frac{1}{2}$: **path-dependent** $\Rightarrow$ DP fails

Previous approach (BCJ 2019):
Lifts the discrete path to a $J{=}100$-dimensional Markov state — expensive and brittle.



Signature approach

Use only $\eta_{d,N} \ll J$ features
(e.g., depth 2: **3 features**)

The Path Signature

Definition

For a continuous path $X : [0, T] \to \mathbb{R}^d$, the **signature** $X^{<\infty}$ collects all iterated integrals:

$$\langle i_1 \cdots i_k,\, X_{0,t}^{<\infty} \rangle \;=\; \int_{0<t_1<\cdots<t_k<t} \mathrm{d}X_{t_1}^{i_1} \cdots \mathrm{d}X_{t_k}^{i_k}, \quad i_j \in \{1, \ldots, d\}.$$

Key properties

1. **Universality**: any continuous functional of X can be approximated by a *linear* functional of $X^{<\infty}$ (Stone–Weierstrass on rough-path space).
2. **Shuffle product**: for group-like elements $g \in G(V)$, multiplying linear functionals corresponds to the shuffle product of their indices:

$$\langle l_1, g \rangle \cdot \langle l_2, g \rangle = \langle l_1 \sqcup\!\sqcup l_2,\, g \rangle$$

Time Augmentation

Adding the Time Variable

Define the augmented path $\widehat{X}_t = (t, X_t) \in \mathbb{R}^{1+d}$.

Why do we need this?

- It injects the time axis into the signature. Since time is strictly increasing, the path no longer has "tree-like excursions".
- $\Rightarrow$ The signature becomes **strictly unique** for each path.
- Letter $\mathbf{1}$ = time component ; letters $\mathbf{2}, \ldots, d{+}1$ = path components.
- $\widehat{X}^{<\infty}$ encodes any *causal* functional of (t, X_t).

Shuffle linearization (key identity for optimization)

Using the shuffle product property, any polynomial P applied to a linear functional of the signature can be rewritten as a **single linear functional**:

$$P(\langle l, g \rangle) = \langle P^{\text{ш}}(l),\, g \rangle$$

Log-Signature: Compressed Representation

Definition

$$L_t^{<\infty} \;=\; \log^{\otimes}(X_t^{<\infty}) \;=\; \sum_{k=1}^{\infty} \frac{(-1)^{k+1}}{k} (X_t^{<\infty} - \mathbf{1})^{\otimes k} \;\in\; \mathfrak{g}(V) \text{ (free Lie algebra)}$$

Dimensionality comparison

	$d=2$		$d=3$	
N	$\sigma_{d,N}$	$\eta_{d,N}$	$\sigma_{d,N}$	$\eta_{d,N}$
1	2	2	3	3
2	6	**3**	12	**6**
3	14	**5**	39	**14**
4	30	8	120	32
5	62	14	363	80

Why use the log-signature for DNNs?

- Removes linear redundancies present in the full signature.
- Dimension formula (Witt / Möbius):

$$\eta_{d,N} = \sum_{n=1}^{N} \frac{1}{n} \sum_{k|n} \mu(n/k)\, d^k$$

- For d=2, N=2: only **3 features** (vs. 100 features for BCJ).

Randomization: Smoothing the Stopping Problem (1/2)

Definition 4.1 — Randomized stopping time

For a *continuous* stopping policy $\theta : \Lambda_T \to \mathbb{R}$ and $Z \sim \mathrm{Exp}(1)$ independent of X:

$$\tau_\theta^r := \inf\left\{ t \geq 0 \ : \ \int_0^{t\wedge T} \theta(\widehat{X}|_{[0,s]})^2 \, \mathrm{d}s \ \geq \ Z \right\}$$

Why is this crucial ?

- Classic stopping times use a discontinuous indicator function (e.g. "stop if price $> K$").
- Discontinuous functions cannot be optimized easily with modern Deep Learning tools (pas de gradient).
- By making the stopping time **randomized** via an intensity θ, the expected payoff becomes completely **differentiable** with respect to θ.
- This allows the use of backpropagation and gradient-based optimizers (like SGD or Adam).

Randomization: Smoothing the Stopping Problem (2/2)

Proposition 4.4 — Smooth expected payoff

With $Z \sim \mathrm{Exp}(1)$, the expected payoff becomes:

$$\mathbb{E}\big[Y_{\tau_\theta^r \wedge T}\big] = \mathbb{E}\left[\int_0^T Y_t\, \theta_t^2\, e^{-\int_0^t \theta_s^2\, \mathrm{d}s}\, \mathrm{d}t + Y_T\, e^{-\int_0^T \theta_s^2\, \mathrm{d}s}\right]$$

Proposition 4.2 — No loss of value

Smoothing the problem does not change the optimal value:

$$\sup_{\theta \in \mathcal{T}} \mathbb{E}\big[Y_{\tau_\theta^r \wedge T}\big] \;=\; \sup_{\tau \in \mathcal{S}} \mathbb{E}[Y_{\tau \wedge T}]$$

Intuition: Any deterministic stopping time τ can be approximated by a sequence of randomized times $(\tau_{\theta_n}^r)$ where θ_n spikes to infinity at τ. The term $G_t := e^{-\int_0^t \theta_s^2\, \mathrm{d}s} = \mathbb{P}(\tau_\theta^r > t \mid X)$ acts as a **survival process**.

Two Families of Signature Stopping Policies

1 — Linear signature policies

$$\theta_l(\widehat{X}|_{[0,t]}) = \langle l,\ \widehat{X}^{<\infty}_{0,t} \rangle$$

Proposition 5.5:

$$\sup_{l} \mathbb{E}[Y_{\tau_l \wedge T}] = \sup_{\tau \in \mathcal{S}} \mathbb{E}[Y_{\tau \wedge T}]$$

Advantage: For polynomial Y, reduces to optimizing over the *expected signature* $\mathbb{E}[\widehat{X}^{\leq N}_{0,T}]$ (deterministic object).

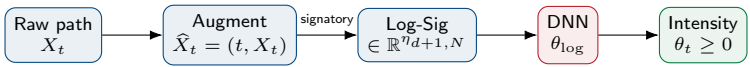
2 — Deep log-signature policies

$$\theta(\widehat{X}|_{[0,t]}) = \theta_{\log}\left(\log^{\otimes} \widehat{X}^{\leq N}_{0,t}\right)$$

Theorem 1.2:

$$\sup_{\theta \in \mathcal{T}_{\log}} \mathbb{E}[Y_{\tau^r_\theta \wedge T}] = \sup_{\tau \in \mathcal{S}} \mathbb{E}[Y_{\tau \wedge T}]$$

Advantage: DNN ($\theta_{\log}$) handles nonlinearities; log-signature gives compressed, non-redundant features.



Linearization via the Shuffle Exponential

When Y is polynomial in X, the full optimization reduces to:

Corollary 6.6 — Deterministic optimization

For $d = 1$, $Y_t = G(X_t) + \int_0^t L(X_s)\,\mathrm{d}s$ (polynomials G, L):

$$V^* \;=\; \lim_{N\to\infty} \sup_{l} \left\langle \big(\exp^{\sqcup\!\sqcup}(-(l \sqcup\!\sqcup l)\cdot \mathbf{1}) \sqcup\!\sqcup G'(\mathbf{2})\big)\mathbf{2} \;+\; \big(\exp^{\sqcup\!\sqcup}(-(l \sqcup\!\sqcup l)\cdot \mathbf{1}) \sqcup\!\sqcup L(\mathbf{2})\big)\mathbf{1},\; \mathbb{E}\Big[\widehat{X}_{0,T}^{\leq N}\Big] \right\rangle$$

Key algebraic steps (each uses the shuffle identity):

1. $\langle l, \widehat{X}^{<\infty}\rangle^2 = \langle l \sqcup\!\sqcup l,\, \widehat{X}^{<\infty}\rangle$
2. $\int_0^t \langle l \sqcup\!\sqcup l, \widehat{X}_{0,s}^{<\infty}\rangle\,\mathrm{d}s = \langle (l \sqcup\!\sqcup l)\cdot \mathbf{1},\, \widehat{X}_{0,t}^{<\infty}\rangle$
3. $\exp(-\langle\cdot\rangle) \approx \langle \exp^{\sqcup\!\sqcup}(\cdot),\, \pi^{\leq N}(\cdot)\rangle$
4. $\mathbb{E}[\text{linear in } \widehat{X}^{<\infty}] = \text{linear in } \mathbb{E}[\widehat{X}^{<\infty}]$

Practical limitation

In practice, this linearized method performs **worse** than the Deep Log-Signature approach due to the truncation error of the shuffle exponential.

Training Algorithm

Loss function

Given M sample trajectories $(\widetilde{X}_j^{(m)}, \widetilde{Y}_j^{(m)})$:

$$\ell(\theta_{\log}) = -\frac{1}{M}\sum_{m=1}^{M}\left[\widetilde{Y}_0^{(m)} + \sum_{j=0}^{J-1} G^{(m)}(j)\,\Delta\widetilde{Y}_{j+1}^{(m)}\right]$$

with $G^{(m)}(j) = \exp\Big(-\sum_{i=0}^{j}\theta_{\log}(\widetilde{L}_i^{(m)})^2\Big)$

Lower-bound estimation

Evaluate learned θ^* on fresh paths $\Rightarrow$ **low-biased Monte Carlo estimate** of V^*

Neural network architecture

- Input: $\widetilde{L}_j \in \mathbb{R}^{\eta_{d+1,N}}$
- 2 hidden layers, ReLU
- Output: scalar intensity $\theta_t \geq 0$

Computational pipeline

1. Simulate M paths of X.
2. Compute prefix log-signatures $(\widetilde{L}_j)_j$ via `signatory`.
3. Train DNN on ℓ via Adam.

Key: log-signature dimension $\eta_{d+1,N}$ does **not** depend on J (time steps).

Code Sketch — Randomized Stopping Loss

```python
def randomized_stopping_loss(theta, payoffs):
    # theta  : (Batch, Time) learned intensity
    # payoffs: (Batch, Time) discounted process Y_t

    cumsum_sq = torch.cumsum(theta ** 2, dim=1)
    G         = torch.exp(-cumsum_sq) # survival proba

    dY        = payoffs[:, 1:] - payoffs[:, :-1]
    E_payoff = payoffs[:, 0] + (G * dY).sum(dim=1)

    return -E_payoff.mean()    # minimise negative payoff
```

Forward pass

- DNN maps each log-signature to θ_j
- G_j computed cumulatively (no loop)
- Gradient flows through G_j and θ_j

Implementation notes

- `signatory.logsignature` used
- Detach tensors to break C++ links
- `OMP_NUM_THREADS=1` avoids CPU deadlocks

Example 1 — Optimal Stopping of fBm ($Y \equiv X^H$)

Setup: $T = 1$, $J = 100$, payoff $Y_t = X_t^H$; 2^{21} training paths.

	Linear ($\mathcal{T}_{\text{sig}}$)			**Deep Log-Sig** ($\mathcal{T}_{\log}$)					**BCJ'19**	
H	N=3	N=4	N=5	N=1	N=2	N=3	N=4	N=5	low	up
0.1	0.760	0.858	0.916	0.929	**1.030**	1.038	1.042	1.043	1.048	1.049
0.2	0.468	0.523	0.553	0.584	**0.637**	0.646	0.649	0.651	0.658	0.659
0.3	0.257	0.285	0.293	0.329	**0.355**	0.361	0.363	0.364	0.369	0.380
0.5	0.000	0.000	0.000	0.000	**0.000**	0.000	0.000	0.000	0.000	0.005
1.0	0.296	0.307	0.309	0.395	**0.395**	0.395	0.395	0.395	0.395	0.395

Key observations

- Deep log-sig at N=2 (3 features) achieves BCJ-comparable values
- H=1: exact value ≈ 0.395 recovered
- H=0.5: value is correctly 0

Linear method

- Underperforms for all H, N
- Exponential shuffle approximation error dominates the training

Example 2 — American Put in a Rough Electricity Market

Model (Bennedsen 2017):

$$S_t = e^{X_t}, \quad X_t = x_0 + \frac{\sigma}{c_\alpha} \int_{-\infty}^{t} (t-s)^\alpha e^{-\lambda(t-s)} \, \mathrm{d}W_s, \quad \alpha \in (-\tfrac{1}{2}, \tfrac{1}{2})$$

$\alpha = -0.4 \Rightarrow$ *rough* paths (γ-Hölder, $\gamma < 0.1$)

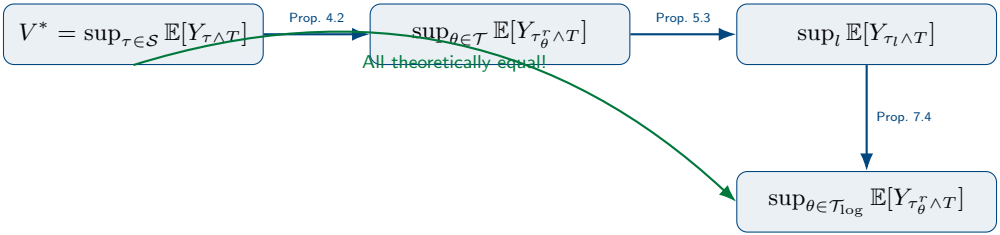
Payoff: $Y_t = e^{-rt}(K - S_t)^+$; Parameters: x_0=100, $\alpha = -0.4$, λ=0.02, σ=0.2.

Strike K	N=1	N=2	N=3	N=4	EUR
80	3.923	4.062	4.076	4.073	0.516
90	9.841	10.185	10.206	10.228	2.139
100	17.753	**18.279**	18.341	18.363	5.770
110	26.679	27.352	27.397	27.423	11.580

Observations

- Values improve significantly from N=1 to N=2; quickly saturate for $N \geq 3$.
- All American prices $\gg$ European prices (strong early-exercise premium).

Theoretical Results at a Glance



Main assumptions

- X is a continuous geometric random rough path
- $\mathbb{E}[\|Y\|_\infty] < \infty$
- $Z \sim \mathrm{Exp}(1)$ independent of X

Comparison with BCJ (2019)

- BCJ: DNN per time step, lifted 100-dim Markov state
- **Here:** *Single* DNN on few log-sig features
- Rigorous theory vs. heuristic Markov lifting

Conclusion & Perspectives

What the paper delivers

1. **Rigorous theory**: optimal stopping value equals the supremum over signature policies, for any continuous rough path.
2. **Linearization**: reduces to deterministic optimization over expected signature.
3. **Practical algorithm**: train a DNN on prefix log-signatures using a smooth loss.

Strengths

- Model-free, dimension independent of time grid size J
- Mathematically grounded
- Numerically highly competitive

Open directions

- Multi-dimensional underlyings ($d \gg 1$)
- Extension to BSDE settings